

ADVANCED OPERATING SYSTEMS

Lab 3 — Locking and lockless reads

Vittorio Zaccaria

Politecnico di Milano

Overview

What you will build

A warm-up on a provided module, then one module of your own, rewritten twice.

- **Warm-up — atomics.** Run a racy counter and an atomic one; explain why only one of them ends at zero.
- **Checkpoint 0 — the unsafe list.** Copy the scaffold: a shared list, three readers and one writer, with no synchronization. Read it and measure it.
- **Task 1 — spinlock.** Make the same file correct with one spinlock.
- **Task 2 — RCU.** Remove the lock: readers traverse without locking, the writer copies, publishes and reclaims.

Each step ends with a build and a **checkpoint** in QEMU with four vCPUs. Answer the questions on each checkpoint slide before moving on.

The lab answers two questions: what does it take to keep a two-field invariant correct under concurrency, and what does it cost the readers?

Set up QEMU with four vCPUs

A one-vCPU guest can interleave threads but cannot run them at the same time. Use either frontend from Lab 1 and boot QEMU with four virtual CPUs.

With the host-terminal frontend:

```
# Host
T=amd64 E=full make dev-up
T=amd64 E=full make dev-build
T=amd64 E=full SMP=4 make dev-run
```

With the browser frontend, use **Terminal → Run Build Task**, then run the `AOS: Run QEMU (4 vCPUs)` task.

Inside the guest, `grep -c '^processor' /proc/cpuinfo` must print 4. Exit QEMU with `Ctrl-]`, then `x`.

Set up the working copy

The starter is `modules/lab-3-th-rcu-scaffold`. Do not edit it: copy it once and work only on the copy.

```
# Host
cp -R modules/lab-3-th-rcu-scaffold modules/lab-3-th-list-work
```

`make dev-build` finds the new directory and installs `/modules/lab-3-th-list-work.ko` in the guest. Edit `modules/lab-3-th-list-work/module.c` with `make dev-vi F=modules/lab-3-th-list-work/module.c` or from the browser Explorer. Rebuild after every change, then boot with `SMP=4` again.

All timings come from QEMU: compare them only within the same run.

Warm-up — atomics

W · Task: run the counter demo

The provided counter module starts two kthreads together. One adds `+1`, the other `-1`, the same number of times: the expected result is 0.

- `variant=0` (**racy**): each iteration loads `shared_value`, calls `cpu_relax()` to widen the window, and stores `value + delta`. Loads and stores use `READ_ONCE` / `WRITE_ONCE`.
- `variant=1` (**atomic**): each iteration is one `atomic64_add`.

Read `counter_thread` and `counter_demo_init`, then run both variants:

```
# Guest, four vCPUs
insmod /modules/lab-3-th-atomics.ko variant=0
rmmod module
insmod /modules/lab-3-th-atomics.ko variant=1
rmmod module
dmesg | grep counter_demo
```

W · Checkpoint

Expected: `variant=racy` has a non-zero `result` that changes from run to run; `variant=atomic` always has `result=0`. Example from one run:

```
counter_demo variant=racy   cpus=4 ... result=1078378 expected=0 ...
counter_demo variant=atomic cpus=4 ... result=0 expected=0 ...
```

Do not compare `elapsed_ns`: `cpu_relax()` slows the racy loop on purpose.

Questions.

1. Every load and every store in the racy loop is a single instruction. How can the result still be wrong?
2. `READ_ONCE` / `WRITE_ONCE` do not fix the race. What do they do, and what happens to the generated loop if you remove them?
3. Each list node in the next part has two fields, `value` and `check`, that must stay consistent. Why can an atomic operation not protect that?

W · Optional: look at the generated code

After `make dev-build`, disassemble the two loops on the host:

```
# Host
D='objdump -d --no-show-raw-insn /repo/modules/lab-3-th-atomics/module.o'
F='/<counter_thread>:/,/^$/p'
docker exec lkp-dev-amd64 sh -c "$D" | sed -n "$F" | grep -B3 -A5 pause
docker exec lkp-dev-amd64 sh -c "$D" | sed -n "$F" | grep -B1 -A3 'lock add'
```

Browser frontend: run the same commands in the code-server terminal without `docker exec lkp-dev-amd64`.

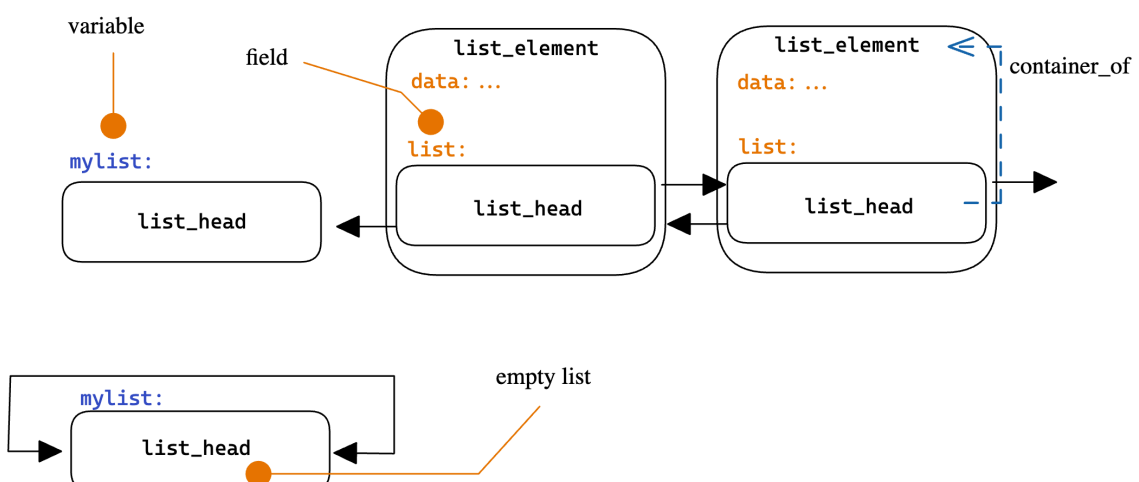
Find the load and the store of `shared_value` in the racy loop, and the single `lock add` of the atomic loop.

Checkpoint 0 — The unsafe list

C0 · The shared list

Each node embeds its `struct list_head` and must satisfy the invariant `node->check == ~node->value`:

```
struct benchmark_node {
  u64 value;
  u64 check;
  struct list_head link;
};
```



C0 · Read the scaffold

Open `modules/lab-3-th-list-work/module.c` and find:

FUNCTION	WHAT IT DOES
`initialize_list`	builds `list_nodes` nodes with `value=i`, `check=~i`
`inspect_list`	one traversal; counts nodes that break the invariant
`reader_thread`	calls `inspect_list` until the deadline, counts `traversals`
`update_list`	the writer's step: first node, `value`, `udelay(1)`, `check`
`writer_thread`	calls `update_list`, counts `updates`, sleeps `writer_pause_us`
`list_demo_init`	creates the threads, starts them, waits, prints one line
`list_demo_exit`	frees the list

Two completion patterns order the threads: `start_gate` (one `complete_all` releases every thread at once) and each `context->done` (one `complete` per thread, one `wait_for_completion` in init). `insmod` returns only after every thread has finished.

C0 · Parameters and output

```
readers=1..8           reader kthreads (default 3)
duration_ms=3000       measurement interval
writer_pause_us=1000   pause between updates
list_nodes=64          list length
```

`list_demo_init` prints one line, after all threads have exited:

```
list_demo variant=<label> cpus=<n> readers=<r> duration_ms=<d>
          writer_pause_us=<p> traversals=<t> updates=<u> violations=<v>
```

(one line in `dmesg`, wrapped here). The scaffold prints `variant=unsafe`. Your two versions must keep this format and change only the label.

C0 · Checkpoint

```
# Guest, four vCPUs
insmod /modules/lab-3-th-list-work.ko \
  readers=3 duration_ms=1000 writer_pause_us=1000
rmmod module
dmesg | grep 'list_demo variant=unsafe' | tail -1
```

Expected: `violations` greater than 0 (one run: `violations=25402`).

Questions.

1. Between which two statements of `update_list` is the first node inconsistent? Why is a run with `violations=0` not a proof of correctness?

2. Readers and writer update their counters with plain `++` , and init sums them with plain reads. Why is that safe?

3. Why are `reader_context` and `writer_context` marked `____cacheline_aligned_in_smp` ?

Task 1 — Spinlock

T1 · Task: protect the list with one lock

Modify `modules/lab-3-th-list-work/module.c`:

- define one global spinlock for the list;
- `inspect_list` holds it for the **whole** traversal;
- `update_list` holds it across the emptiness check and both stores; keep `udelay(1)` inside the critical section, so its cost stays visible;
- `cond_resched()` in the reader loop and the writer's `usleep_range()` stay **outside** the lock: code holding a spinlock must not sleep;
- change the label in the final `pr_info` to `variant=spinlock`.

NEED	API
header	`<linux/spinlock.h>`
static lock, initially unlocked	`DEFINE_SPINLOCK(name)`
enter / leave the critical section	`spin_lock(&name)`, `spin_unlock(&name)`

No interrupt handler touches the list, so the plain `spin_lock` is enough.

T1 · Checkpoint

```
# Guest
insmod /modules/lab-3-th-list-work.ko \
  readers=1 duration_ms=1000 writer_pause_us=1000
rmmod module
insmod /modules/lab-3-th-list-work.ko \
  readers=3 duration_ms=1000 writer_pause_us=1000
rmmod module
dmesg | grep 'list_demo variant=spinlock' | tail -2
```

Expected: `violations=0` in both lines; total `traversals` with three readers is **lower** than with one. Example: 941737 → 285492.

Questions.

1. Readers never modify the list. Why do three readers complete fewer traversals in total than one?
2. Name the calls of the scaffold that must stay outside the critical section, and say why.
3. What would a reader observe if the writer released the lock between the two stores?

Task 2 — RCU

T2 · Idea: copy, publish, reclaim

RCU readers take no lock. A read-side critical section (`rcu_read_lock` … `rcu_read_unlock`) does not exclude the writer: it only guarantees that a node the reader can reach is not **freed** before it leaves.

So the writer never modifies a node that readers can see:

1. **copy**: allocate a new node and fill it completely while it is private;
2. **publish**: link it into the list in place of the old one, with one pointer store that readers observe all at once;
3. **reclaim**: free the old node only after a **grace period**, when every reader that might still stand on it has left its critical section.

RCU requires that updaters never overlap. Here there is exactly one writer thread, so the update side needs **no lock**. With several writers, a lock among writers would still be needed; readers would still take none.

T2 · Task: lockless readers, copy-publish-reclaim writer

Starting from your spinlock version:

- remove the spinlock completely and add a `struct rcu_head rcu;` member to `struct benchmark_node`;
- write a callback `void free_node_rcu(struct rcu_head *head)` that frees the node containing `head`;
- `inspect_list`: traverse inside one read-side critical section with the RCU list iterator;
- `update_list` becomes `bool update_list(void)`:
    1. return `false` if the list is empty or the allocation fails;
    2. allocate a replacement for the **first** node and set `value = old value + 1`, `check = ~value`; keep `udelay(1)` before publishing;
    3. replace the old node with it in the list;
    4. hand the old node to RCU with your callback; return `true`;
- `writer_thread` increments `updates` only when `update_list` returns `true`;
- `list_demo_init` calls `rcu_barrier()` after waiting for the writer, before printing and returning;

- change the label to `variant=rcu`.

T2 · Useful APIs

NEED	API
headers	`<linux/rculist.h>` , `<linux/rcupdate.h>`
read-side critical section	`rcu_read_lock()` , `rcu_read_unlock()`
traverse an RCU-protected list	`list_for_each_entry_rcu(pos, head, member)`
first node	`list_first_entry(head, type, member)`
allocate the replacement	`kmalloc(size, GFP_KERNEL)`
publish a replacement	`list_replace_rcu(&old->link, &new->link)`
free after a grace period	`call_rcu(&old->rcu, free_node_rcu)`
node from its `rcu_head`	`container_of(head, struct benchmark_node, rcu)`
wait for the RCU callbacks queued so far	`rcu_barrier()`

`call_rcu` returns immediately; RCU runs the callback after a grace period, in softirq context: the callback must not sleep (`kfree` is fine). `kmalloc` with `GFP_KERNEL` may sleep: the writer is not inside a read-side critical section, so it may call it.

The callback is **module code**. `rcu_barrier()` returns after every RCU callback queued so far has run: without it, a callback could run after `rmmod`.

T2 · Inside `list_replace_rcu`

`include/linux/rculist.h` replaces `old` with `new` in five pointer assignments:

```
new->next = old->next;                                  /* private      */
new->prev = old->prev;                                  /* private      */
rcu_assign_pointer(list_next_rcu(new->prev), new);      /* publication  */
new->next->prev = new;                                  /* plain store  */
old->prev = LIST_POISON2;                               /* catch misuse */
```

- `rcu_assign_pointer` stores `prev->next = new` with release semantics: a reader that follows the new pointer also sees `value`, `check` and `next`. `list_for_each_entry_rcu` reads each `next` with `rcu_dereference`.
- RCU readers only move forward, so `prev` needs no ordering.
- `old->next` stays valid: a reader standing on `old` continues normally.

T2 · Checkpoint

```
# Guest
insmod /modules/lab-3-th-list-work.ko \
  readers=1 duration_ms=1000 writer_pause_us=1000
rmmod module
insmod /modules/lab-3-th-list-work.ko \
  readers=3 duration_ms=1000 writer_pause_us=1000
rmmod module
dmesg | grep 'list_demo variant=rcu' | tail -2
```

Expected: `violations=0` in both lines; total `traversals` with three readers is **higher** than with one. Example: 895864 → 2538763.

Questions.

1. Why does the writer publish a new node instead of updating `value` and `check` of the old one?
2. Why can the old node not be freed with `kfree` right after `list_replace_rcu`? Why is `rcu_barrier()` mandatory before the module can be unloaded?
3. Why is no lock needed on the update side here? When would you need one?

Wrap-up

Compare the variants

Example from one verified one-second run on four QEMU vCPUs:

VARIANT	READERS	TRAVERSALS	UPDATES	VIOLATIONS
unsafe	3	2,908,844	748	25,402
spinlock	1	941,737	753	0
spinlock	3	285,492	671	0
RCU	1	895,864	803	0
RCU	3	2,538,763	797	0

Your numbers will differ; the trends should not. Unsafe is fast but wrong; spinlock is correct but readers serialize; RCU is correct and reads scale, paying with an allocation and a deferred free per update.

Optional: rerun the RCU version with `writer_pause_us=0` (a write-heavy workload) and compare `updates` and `traversals` with your previous run.

Done when

- the unsafe scaffold reports `violations` greater than 0;
- `variant=spinlock` reports `violations=0` with one and three readers, and fewer total traversals with three;
- `variant=rcu` reports `violations=0` with one and three readers, and more total traversals with three; the module contains no lock;
- `modules/lab-3-th-rcu-scaffold` is unchanged;
- after the last `rmmod module`, `cat /proc/modules` lists no `module`, and `dmesg | grep -E 'BUG:|WARNING:|Oops:'` prints nothing.

Exit QEMU, then stop the container: `T=amd64 E=full make dev-down` (host terminal) or `make vscode-down` (browser).